Sample for Quiz 3

You may use your notes and the course website.
You may not use Google, any other website, or an LLM.

Load the `tidyverse` package!

1. Apply `str_c` function on `x <- c("Sam", "John")` and `y <- c("Sampson", "Johnson")` into `Sam Sampson and John Johnson`.

```
x <-  c("Sam", "John")
y <- c("Sampson", "Johnson")

str_c(x,y, sep = " ", collapse = " and ")
```

```
## [1] "Sam Sampson and John Johnson"
```

Recall that **Metacharacter**s in *regular expressions* are special characters that have a specific meaning and are used to define search patterns. Here is a brief summary of some common metacharacters:

1. `.` **(Dot):** Matches any single character except newline (`\n`).
2. `^` **(Caret):** Matches the start of a string or line.
3. `$` **(Dollar):** Matches the end of a string or line.
4. `*` **(Asterisk):** Matches zero or more occurrences of the preceding character.
5. `+` **(Plus):** Matches one or more occurrences of the preceding character.
6. `?` **(Question Mark):** Matches zero or one occurrence of the preceding character.
7. `[]` **(Character Class):** Matches any single character inside the brackets.
8. `|` **(Pipe):** Acts as an OR operator, allowing matching of either expression on its left or right side.
9. `()` **(Parentheses):** Creates a capture group for extracting substrings or applying quantifiers.
10. `\` **(Backslash):** Escapes metacharacters or introduces special sequences (e.g., `\\d` for digit, `\\w` for word character).

These metacharacters provide powerful tools for constructing complex search patterns in regular expressions.

2. First, produce the string `\.?3^!*7+[` and then convert it to `0123456789`. Next, convert it to `[+7*!^3?.\`

```
x1 <- "\\.?3^!*7+["
x2 <- str_replace(x1, "\\\\\\.\\?3\\^!\\*7\\+\\[", "0123456789")
x2
```

```
## [1] "0123456789"
```

```
x3 <- str_replace(x1, "(\\\\)(\\.)(\\?)(3)(\\^)(!)(\\*)(7)(\\+)(\\[)", "\\10\\9\\8\\7\\6\\5\\4\\3\\2\\1
x3
```

```
## [1] "[+7*!^3?.\\"
```

```
writeLines(x3)
```

```
## [+7*!^3?.\
```

3. In a given vector of strings, find any seven- or eight-letter words that end with `sion`, `tion` or `tory`; starting with `p` or `f` or `d` (little or capital). Test on `c("Passion", "satisfactory", "factory", "satisfaction", "devotion", "pa12tion")`.

```
x <- c("Passion", "satisfactory", "fActory", "satisfaction", "devotion", "pa12tion", "pansiOn")
str_replace_all(x, "^((?i)[pfd])[a-z]{2,3}(sion|tion|tory)$", "X")
```

```
## [1] "X"            "satisfactory" "fActory"      "satisfaction" "X"
## [6] "pa12tion"     "pansiOn"
```

```
str_replace_all(x, "^[pfdPFD][a-z]{2,3}(sion|tion|tory)$", "X")
```

```
## [1] "X"            "satisfactory" "fActory"      "satisfaction" "X"
## [6] "pa12tion"     "pansiOn"
```

4. In a given vector of strings, find any string starting with any two letters followed by a comma, followed by at least one space and at most two spaces, and ending with any five digits.

```
x <- c("MD 21228", "WA, 76731", "VA: 34124", "FL, 21225", "ca,  45691")

str_replace_all(x, "^((?i)[a-z]{2}),\\s{1,2}\\d{5}$", "X")
```

```
## [1] "MD 21228"  "X"         "VA: 34124" "X"         "X"
```

```
str_replace_all(x, "^[a-zA-Z]{2},\\s{1,2}\\d{5}$", "X")
```

```
## [1] "MD 21228"  "X"         "VA: 34124" "X"         "X"
```

5. In a given vector of strings, find any string in which at least three consonants are in a row.

```
x2 <- c("HHHonorsion", "DBBELIcious", "Prio5ity", "HIROSHIMA", "Uinicasion", "Cat", "Catsion", "Dog", "
str_replace_all(x2, "(?i)[^aeiou]{3,}", "X")
```

```
##  [1] "Xonorsion"  "XELIcious"  "Prio5ity"   "HIROSHIMA"  "Uinicasion"
##  [6] "Cat"        "Catsion"    "Dog"        "FiniX"      "Extension"
## [11] "Xinger"
```

6. In a given vector of strings, find any string in which at least three consonants are in a row, where the first and third ones are upper case and the second one must be lower case. If there are more than three consonants in a row, we do not require them to be upper or lower.

```
x <- c("H2norsion", "DELIcious", "Priority", "HIROSHIMA", "Uinicasion", "CATti", "CASIONnNbd", "Dog", "
str_replace_all(x, "[^AEIOU][^aeiou][^AEIOU][^aeiouAEIOU]*", "X")
```

```
##  [1] "XXion"   "XIcious" "XX"      "XOSXA"   "UXXon"   "Xi"      "XIOX"
##  [8] "Dog"     "FXX"     "EXion"   "SXer"
```

7. Suppose a given word starts with a consonant. Switch this first letter with the first vowel in the given word.

```
x <- c("DELIcious", "Priority", "HIROSHIMA", "Dog", "Fishing", "Extension", "Singer")
str_replace_all(x, "(^[^aeiouAEIOU])([^aeiouAEIOU]*)([aeiouAEIOU])(.*)", "\\3\\2\\1\\4")
```

```
## [1] "EDLIcious" "irPority"  "IHROSHIMA" "oDg"       "iFshing"   "Extension"
## [7] "iSnger"
```

A simplified explanation of positive and negative lookahead and non-capturing groups with examples:

- **Positive Lookahead:**
  - **Definition:** Positive lookahead is a pattern that matches a group of characters only if they are followed by another specified pattern.
  - **Syntax:** `(?=pattern)`
  - **Example:** Suppose we want to match all occurrences of the word "cat" only if it is followed by the word "dog". The regular expression for this would be `cat(?=dog)`.
- **Negative Lookahead:**
  - **Definition:** Negative lookahead is a pattern that matches a group of characters only if they are not followed by another specified pattern.
  - **Syntax:** `(?!pattern)`
  - **Example:** Suppose we want to match all occurrences of the word "cat" only if it is not followed by the word "dog". The regular expression for this would be `cat(?!dog)`.

The pattern `(?<!pattern)` is a negative lookbehind assertion in regular expressions. It is used to assert that the current position in the string is not **preceded** by a specific pattern.

- **Non-Capturing Groups:**
  - **Definition:** Non-capturing groups are used for grouping parts of a pattern together without capturing the matched text for further use.
  - **Syntax:** `(?:pattern)`
  - **Example:** Suppose we want to match a date in the format "MM/YYYY" but we're only interested in capturing the month. We can use a non-capturing group to match the year without capturing it: `(?:\\d{2})/\\d{4}`.

These concepts are essential for understanding more advanced regular expression patterns and can be very useful in text processing and data extraction tasks. Note that while lookahead assertions and non-capturing groups are useful for organizing and specifying patterns in regular expressions, they do not capture text that can be referred to later using backreferences.

Note that you will not encounter questions that require knowing positive and negative lookaheads in this course. But, understanding them can provide insight into the capabilities of the stringr package. Learning about these advanced features of regular expressions can enhance your understanding of string manipulation and empower you to tackle more complex text processing tasks.

8. In a given vector of strings, find any word starting with a capital letter that does not end with `ing` or `ity` or `sion`. Such words must have at most nine letters.

```
x2 <- c("H2norsion", "DElicious", "Prio5ity", "HIROSHIMA", "Uinicasion", "Cat", "Catsion", "Dog", "Fishi

str_replace_all(x2, "^[A-Z][a-zA-Z]{0,8}(?<!ity)(?<!ing)(?<!sion)$", "XXX")
```

```
##  [1] "H2norsion"  "XXX"        "Prio5ity"   "XXX"        "Uinicasion"
##  [6] "XXX"        "Catsion"    "XXX"        "Fishing"    "Extension"
## [11] "XXX"
```

9. Give a regular expression pattern that can be used to validate an email with the following requirements: contains at least one upper case, one lower case, and one digit! The email ID must be within `[a-zA-Z0-9_]` characters and there is no limitation on the length of the email. However, it must be either `gmail` or `yahoo`.

```
emails <- c("sam34A23@gmail.com", "Su1ut49@yahoo.com", "Su1@ut49@yahoo.com", "Su1-ut49@yahoo.com", "jer
str_detect(emails, "(?=.*[A-Z])(?=.*[a-z])(?=.*\\d)[a-zA-Z0-9_]*@(gmail|yahoo).com")
```

```
## [1]  TRUE  TRUE FALSE FALSE FALSE
```

10. Give a regular expression pattern that can be used to validate a password with the following requirements: contains at least two uppercase letters, exactly one lowercase letter, and includes at least three digits. And, the ID must be at most 10 characters.

```
emails <- c("sA34A2376@gmail.com", "SuBB149@yahoo.com", "Su1@ut49@yahoo.com", "Su1-ut49@yahoo.com", "JE
str_detect(emails, "(?=(.*[A-Z]){2,})(?=(.*[a-z]){1})(?=(.*\\d){3,})\\w{1,10}@(gmail|yahoo).com")
```

```
## [1]  TRUE  TRUE FALSE FALSE  TRUE
```